

Blockchain-Technology Notes for Re-midterm Exams



VII Semester RE MIDTERM TEST
Blockchain Technology (CSE - 4033)

Time Duration: 2 Hours

Date:

Max marks: 30 MARKS

Question No	Topic	Marks	BL	CO
1	DApps are designed for? Choose the right answer a) Execute smart contracts with the business logic in the front-end of a standalone application b) Manage cryptocurrencies only, without any embedded voting system for governance of the blockchain ☒ c) Run applications on a peer-to-peer (P2P) network expanding smart contracts beyond simple value transfer d) Support applications that run on multiple public cloud providers avoiding any vendor lock-in and fraud	1	L1	CO.3
2	Analyze as to how blockchain technology best help securing identity data? a) By eliminating third parties through providing secured-data storage at a user's server b) By encoding all the health data and save it on a private and permissionless blockchain ☒ c) By protecting data that has been submitted on the internet using a cryptographic algorithm d) By providing information personal data without disclosing the actual data that proves it	1	L4	CO.4
3	Public blockchains give an incentive to encourage users to mine blocks and secure the network. Interpret this incentive a) Public blockchains allow users to create tokens to sell on secondary markets. b) Public blockchains do not offer rewards, because they are open source. c) Public blockchains offer cash rewards for running mining nodes. ☒ d) Public blockchains offer rewards for mining in the form of cryptocurrency.	1	L2	CO.4
4	An attacker tries to corrupt the transaction history of a blockchain to be able to spend a token or a cryptocurrency twice. Choose the most likely thing this attacker did? a) The attacker changed the transaction on his node and propagated it in the network. b) The attacker edited the smart contract and recovered investor's cryptocurrency. ☒ c) The attacker gained control of more than 51% of the network's computing power.	1	L1	CO.4

	d) The attacker hard-forked the network and created a new blockchain network.			
5	What is not a classification for a node? a) Full node b) Lightweight node c) Merkle node d) Miner node	1	L1	CO.1
6	Which of the following is not a blockchain component? a) certificate authority b) Ledger c) Wallet d) Hash	1	L1	CO.1
7	Which of the following is true about the bitcoin scripting language? a) support infinite time/memory b) stack based c) Turing complete d) None of the above	1	L1	CO.3
8	What provides the power to the Ethereum Virtual Machine? a) Gas b) Ether c) Both A and B d) None of the above	1	L1	CO.3
9	The alternatives to POW are inferred as. a) Delegated Proof of Stake b) Proof of stake c) Both A and B d) None of the above	1	L2	CO.3
10	Proof of Stake is interpreted as. a) A transaction and block verification protocol b) A certificate needed to use the blockchain c) All of the above d) None of the above	1	L2	CO.3
11	Analyze the need for consensus in blockchain systems. Discuss Proof-of-Work (PoW) and Proof-of-Burn (PoB) algorithms	5	L4	CO.3
12	Compare Multisig, Pay-to-Script-Hash (P2SH), and Proof-of-Burn transactions in terms of usability and security.	5	L2	CO.3
13	Infer the Byzantine Generals problem and its role in blockchain consensus mechanisms.	5	L2	CO.2
14	Define atomic broadcast and discuss its importance in distributed systems	5	L1	CO.1

Note: BL refers to Bloom's Taxonomy Level.

What is a block-chain?

A Block-chain is a digital ledger of transactions distributed across the entire network of computers (or nodes) on block-chain.

A Block-chain uses several technologies like digital signatures, distributed networks encryption/decryption methods to enable block-chain applications

Modelling Faults & Adversaries

1. Double spending Attack.
2. 51% Attack (Majority Attack)
3. Sybil Attack
4. Timejacking
5. Smart Contract vulnerabilities

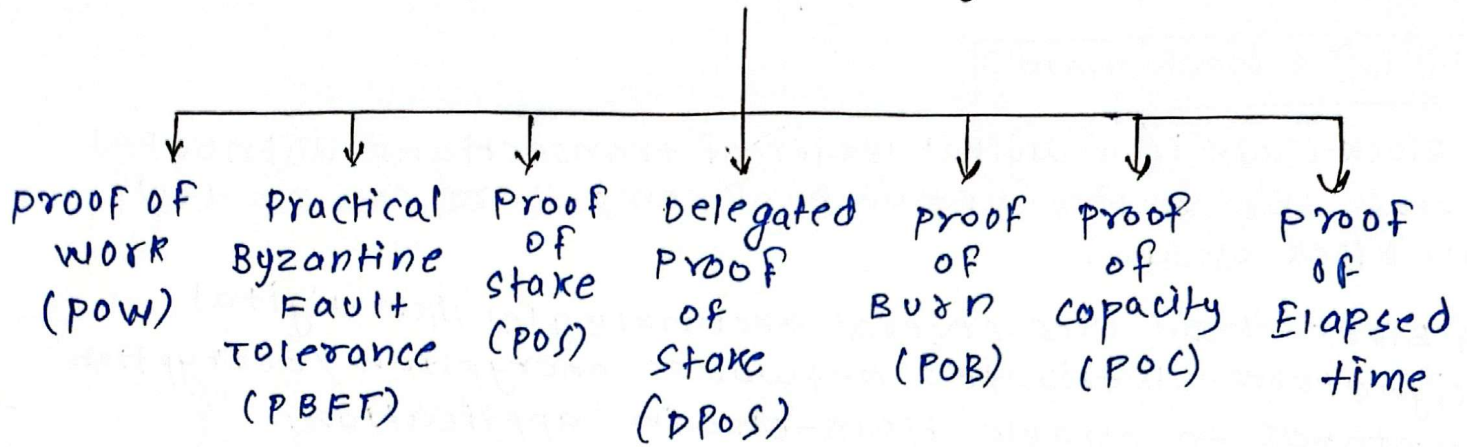
consensus algorithm

- A consensus algorithm allows all peers in a block-chain network to agree on current state of the distributed ledger.
- consensus protocol ensures that only one version of truth is added to blockchain and accepted by all nodes.

key objectives of Block-chain consensus protocol

- Achieving agreement among nodes
- Promoting collaboration and co-operation
- Ensuring equal rights for all nodes
- Requiring mandatory participation of every node in the process.

Types of consensus Algorithms



Proof of Work.

- Miners solve complex mathematical puzzles to add a new block.
- Requires high computational power & Energy
- First used in Bitcoin
- Slow but secure.

Practical Byzantine Fault Tolerance

- Nodes communicate and vote to reach consensus, even if some act maliciously
- Fast and Energy Efficient
- Not ideal for large-scale public blockchains

Proof of Stake

- Validators are chosen on based on the amt of cryptocurrency they hold willing to "stake"
- More efficient than POW
- Reduces expensive hardware
- Used in Ethereum 2.0.

Delegated proof of stake (DPoS)

- Stakeholders vote to elect a few delegates to who validate blocks.
- Fast and Scalable
- More democratic but can become centralized
- Used in networks like EOS.

Proof of Burn (POB)

- users "burn" destroy coins by sending them to an unusable address to earn the right to mine or validate
- shows long term commitment
- slower adoption

Proof of Capacity (Proof of Space)

- uses hard drive space to store solution to puzzle
- More eco-friendly than POW
- The more space you have, higher your chance to mine a block
- used in Burst-coin.

Proof of Elapsed Time (PoET)

- Nodes wait for a randomly chosen time; the first ~~get~~ to finish gets to add the block.
- Requires trusted execution environments
- Fair and Energy Efficient
- used in Hyperledger sawtooth.

Cryptographic Hash Functions

Hash Function Properties

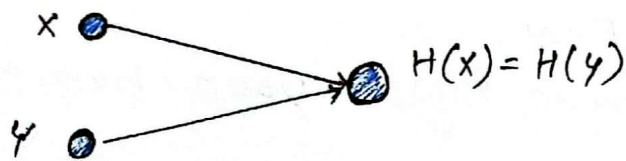
Input can be Any size

Produces Fixed-size outputs (eg: 256 bit, or any sufficient large size)

can compute hash of an n -bit string in $O(n)$ time.

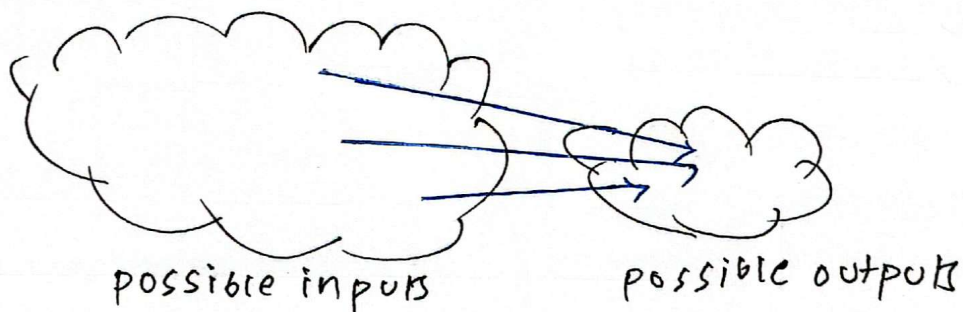
collision resistance Hash Function is collision resistant.

A collision occurs when two distinct inputs produce the same output.



A Hash collision x and y are distinct values where $x \neq y$ yet $H(x) = H(y)$

- 1) collision must exist in a hash fn yet finding them is hard
- 2) input space is infinite
- 3) output space is fixed
- 4) many distinct inputs produce same hash value



Example

$$H(x) = x \bmod^{256}$$

→ Accepts inputs of any lengths

→ Returns a fixed 256-bit output

property-II Hiding

The second property is Hiding. It asserts that if we're given the output of Hash Function $y = H(x)$. There is no feasible way to find out what the input x was.

A Hash Function H is hiding IF: When a secret value r is chosen from probability distribution that has high min-entropy, then given $H(r|x)$ is infeasible to find x .

Example:- coin = $\{H, T\}$
tossed

r if fails

$$\begin{cases} P(H/r) = P(T/H) = \text{output} \\ P(H/r) = P(H/T) = \text{output} \\ \text{if Heads} \end{cases}$$

Easier to predict because probability distribution is not large.

Application:- commitment

Property-III Puzzle Friendliness

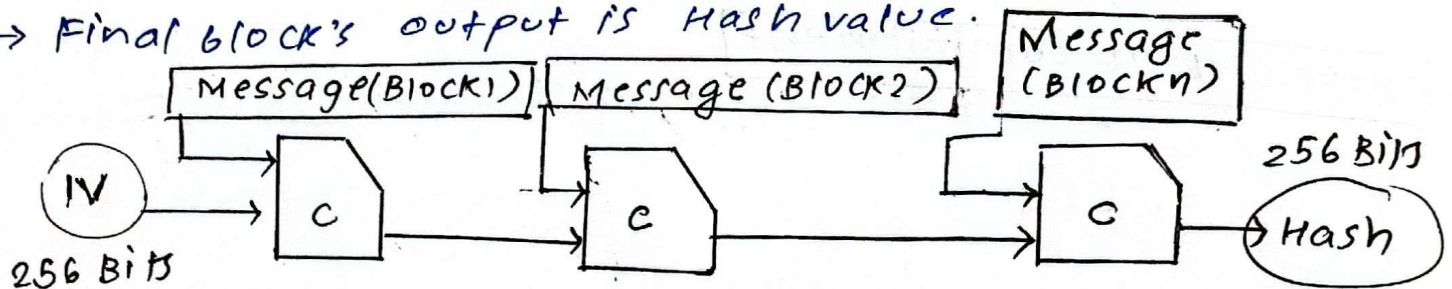
A Hash function H is said to be puzzle-friendly, if every possible n -bit output value of y , if K is chosen from distribution with min High entropy, then it is infeasible to find x such that $H(K||x)=y$ in time significance less than 2^n .

SHA-256

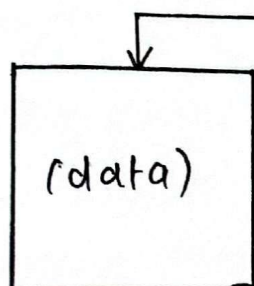
- SHA-256 is a widely used cryptographic hash function (notably in Bitcoin) that processes input of arbitrary length to produce a fixed 256-bit output
- It uses the Merkle-Damgård transform to handle arbitrary lengths

Merkle-Damgård Transform

- Divide Input into blocks of size $m-n$ (512 bits in SHA 256)
- Feeding each block along with the output of previous block, into a compression function takes 768 bit input
- For first block an initialization vector (IV) is used instead of a previous o/p
- Final block's output is Hash value.

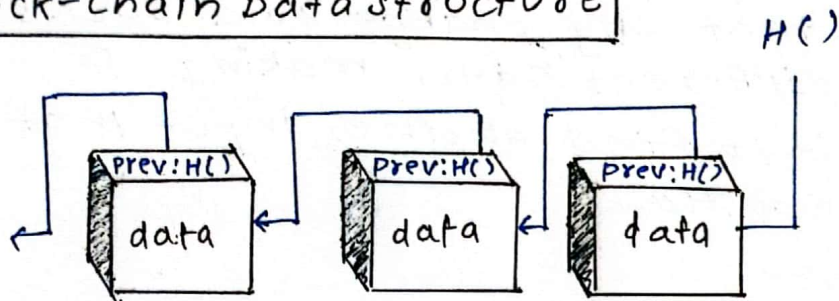


Hash-Pointer and Datastructures



A Hash pointer is simply a pointer to where some information is stored with a cryptographic Hash

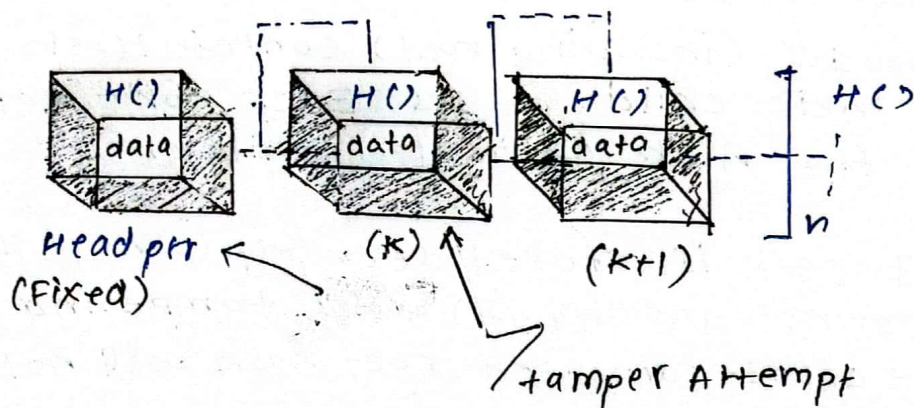
Block-chain Data structure



- A Block-chain is a linked-list where each node, called a block contains
data: information stored in block (eg: transactions in Bitcoin)
- A Hash pointer to the previous block includes memory address of previous block
A cryptographic hash (eg: SHA 256) of previous block data
- Head of Block-chain is a Hash pointer that points to most recent block.

property of Block-chain

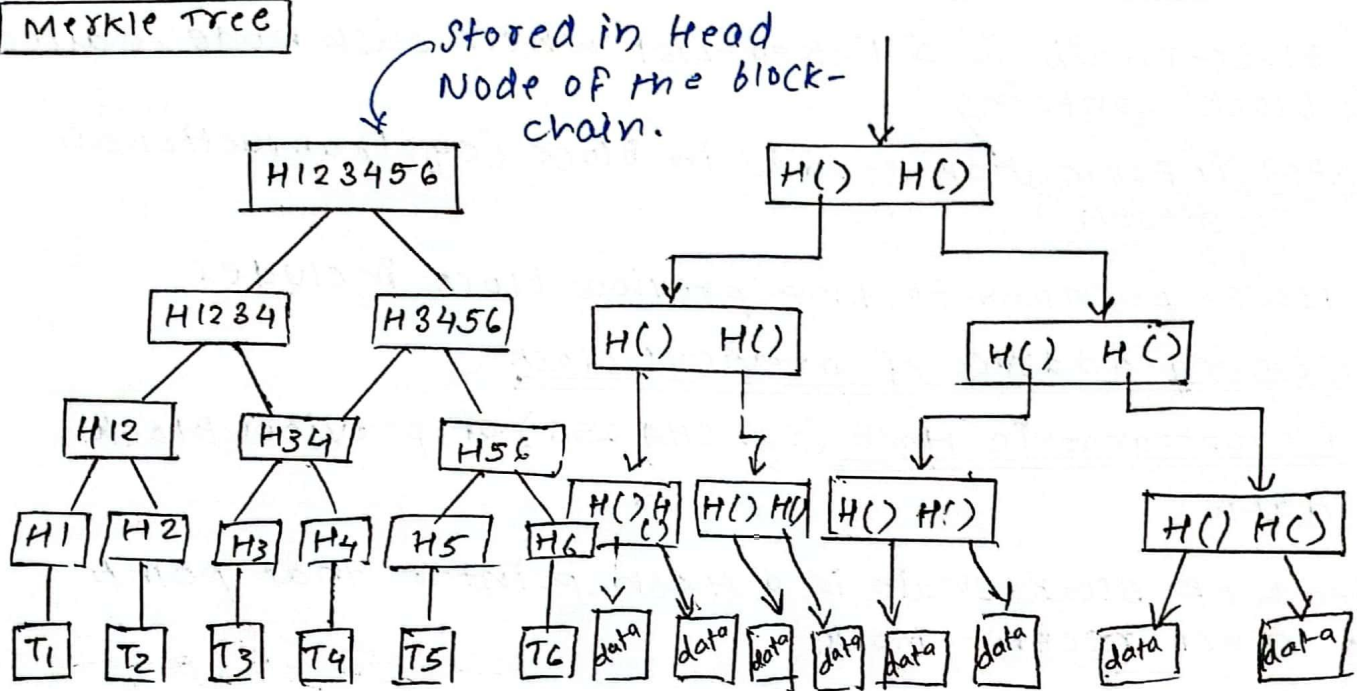
Tamper Evident property



- Since the next block (k+1) contains a hash pointer with the original hash of block (k) the altered data causes a mismatch making the tampering detectable.
- If head's hash pointer is stored secured, the tampering will be detected because the head's hash will no longer match the computed hash of latest block.

- The collision-resistant property of the Hash Function (eg SHA 256) ensures that any change to a block's data produces a different hash, making it statistically impossible to find another input that matches the original hash

Merkle Tree



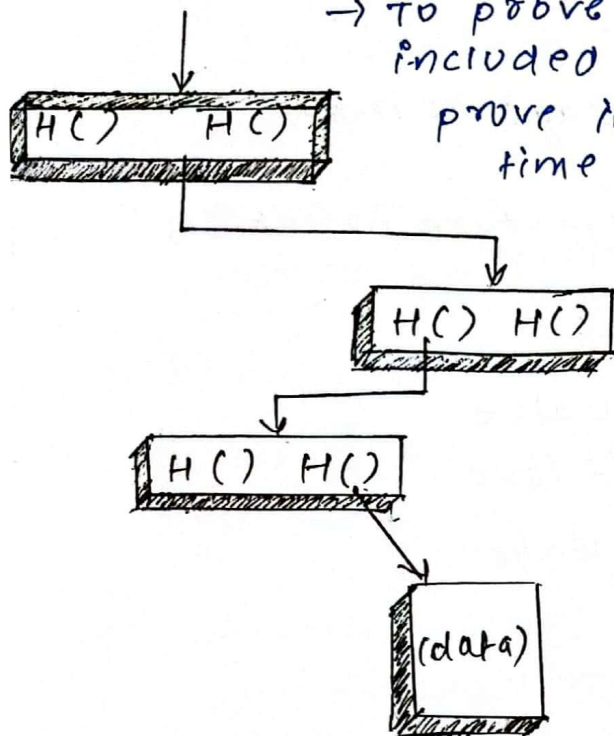
- A merkle tree is a binary tree where:-
 - Leaf nodes store hashes of data blocks
 - Non-Leaf nodes (including root) contain Hash pointers to their child nodes. Each non-leaf node store the hash of its two children
 - If the Root hash is stored securely (where the adversary cannot modify it) any tampering is detectable because computed root hash will not match the stored root hash.

Verification Requires $\log_2(n)$ hashes
time and space complexity $O(\log(n))$

Sorted Merkle Tree

- Tree (Merkle tree) is sorted in a specific order
- sorted structure enables functionality like proving non-membership.

→ To prove that a specific block is not included in a merkle tree, you can prove its not included in logarithmic time and space $O(\log(n))$



proof of membership.

Mechanisms:

1. If want to verify whether a block 'x' lies between a merkle tree if sorted in alphabetical order
2. Verifier checks for block 'x' between A and B by computing the path
3. since B comes immediately after A there exist no 'x' block hence proving the non-membership

Advantages of a sorted merkle tree

- each path involves $\log_2(n)$ nodes for a tree with n leaves
- Tamper evident
- securely stored root hash ensures that tampering is detectable.
- can compute hashes from nodes
- proof of membership

Digital signature

$(SK, PK) := \text{generateKeys}(\text{keysize})$: generate key pair
SK: kept privately to sign messages
PK: to verify signature.

$\text{sig} : \text{sign}(SK, \text{message})$: takes message as input
SK: input
output: signature under SK

$\text{isValid} := \text{verify}(PK, \text{message}, \text{sig})$:

returns Boolean value

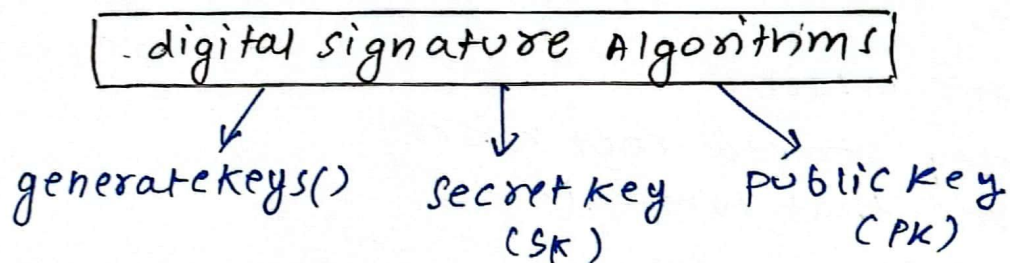
$\text{isValid} \begin{cases} \text{sig is true} \\ \text{o/w false} \end{cases}$

valid signatures must verify

$\text{verify}(PK, \text{message}, \text{sign}(SK, \text{message})) == \text{true}$.

key properties

- only you can sign
- Anyone can verify
- Tied to a specific document



Unforgeability Game

Rules: challenger using (sk) sign valid signature for messages

Adversary is allowed a 'plausible' number of signatures request (eg 1million but not an infeasible number like 2^{80})

Adversary can request signatures on messages of their choice

Game:-

1. Adversary ASKS s_i (request). challenger to sign many messages of his choice
2. challenger signs with a his (sk) key to the messages the Adversary demands
3. Adversary has access to (pk) key, he learns about the (sk) that used to sign many messages
4. He produces his own (sk) by studying the signatures which he requested to get signed
5. Adversary picks a new message (M) and attempts to produce a valid signature
6. pair of (sk, pk) is created, Adversary tests the sk he created if $isvalid == true$
Adversary won the game

Solution For Unforgeability on Digital Signatures:

1. unforgeable if $P(\text{Forging})$ valid signature with access to SK is very small.

Randomness :-

Many signature algorithms including ECDSA are randomized, requiring a good source of randomness for both key generation and signing.

Poor randomness can compromise security.

Message Size Limitation :-

1. Digital signature operate at 256 bits
2. to sign a long message its hashed to produce fixed-length (256 bits)
3. collision property ensures that signing hash is secure as signing original message.

Signing Hash pointer

1. sign hash pointer, it will sign the entire data structure resulting in signing all hash pointers ensuring integrity & authenticity

ECDSA (Elliptic Curve Digital Signature Algo)

1. provides se ECDSA with secp256k1 provides approximately 128 bits of security meaning it is as difficult as performing 2^{128} symmetric key operations
2. considered highly secure

Note: ECDSA can only sign 256-bit messages; therefore Bitcoin hashes messages with SHA 256 before signing, allowing arbitrary lengths of messages to be signed securely.

APPLICATION IN CRYPTOCURRENCY

1. Transaction Authorization
2. Integrity in Block-chains
3. Non-Repudiation.

Bit-coin Addresses

- An Address is a hash of a public key, serving as a pseudonymous identity for receiving and spending funds
- To spend funds, the owner signs a transaction with corresponding private key, proving the control the address

Flexibility

Users can create multiple addresses for different purposes, enhancing privacy and organization.

A simple cryptocurrency

Goofycoin:- Goofy-coin is a minimalistic cryptocurrency with two rules:-

coin creation: Goofy can create coins

coin transfer: The owner can transfer it to someone.

* uses digital signature and Hash pointers to manage ownership and transfers but lacks mechanisms to prevent double (duplicate) spendings.

signed by SK_{Alice}
pay to $PK_{Bob}: H()$

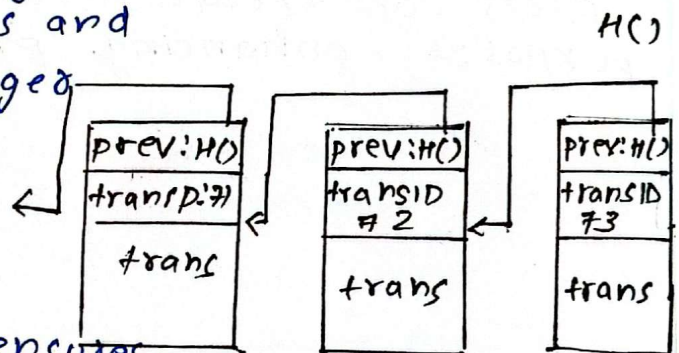
signed by SK_{Goofy}
pay to $PK_{Alice}: H()$

signed by SK_{Goofy}
Create coin [unigecoinid]

Scrooge coin.

Scrooge coin relies on a designated Entity. Scrooge who maintains and publishes an append-only ledger containing the history of all transaction.

A. The append-only property ensures that once a transaction is recorded, it cannot be altered or removed without detection.



By requiring all transactions to be recorded in this public ledger before they are considered valid, Scrooge coin prevents double spending: if a coin is already spent, any attempt to spend it again will be rejected.

• Scrooge's Role :-

- Scrooge digitally signs the hash pointer of the latest block binding the entire chain's integrity.
- The signature ensures that all transaction in the blockchain are endorsed by Scrooge.

Optimization Note :- Multiple transactions can be grouped into a single block for efficiency

Public visibility :- Since the block-chain is publicly visible anyone can check the history of the coin, to see if it's already spent

Scrooge validation :- Scrooge only adds valid transactions to blockchain. Before endorsing a transaction, checks if coin is not spent anywhere else.

Why use Block-chain?

Append-only property

Simplified verification

Consistency Across observers

Block-chain Technology Module 3 - crypto currency.

structure of a Bit-coin Transaction

1. Meta-data (Housekeeping Info)

- size of transaction: How big it is in bytes
- Number of inputs & outputs: How many i/p's are being spent
- Transaction hash (txid): Hash of the whole transaction.
- unique-id: (also used as a hash ptr)
- Lock-time: optional field that specifies the earliest time/block

2. Inputs (What you're spending)

- transaction hash: points to a previous transaction.
- output index: specifies which output of the transaction is being spent
- signature: proves the spender is authorized.

3. outputs (where the money is going)

- value → amount of BTC
- locking script: specifies the conditions to spend o/p
- $\text{sum}(\text{outputs}) \leq \text{sum}(\text{inputs})$
- if $\text{sum}(\text{outputs}) < \text{sum}(\text{inputs})$
leftover is transaction fees

Bitcoin scripts

- A Bit coin script is a small program written in Bitcoin scripting language, which defines the condition that must be met in order to spend a transaction output
- output can be redeemed if a certain script executes successfully
- spender should provide a valid signature

<sig>
 <pubkey>

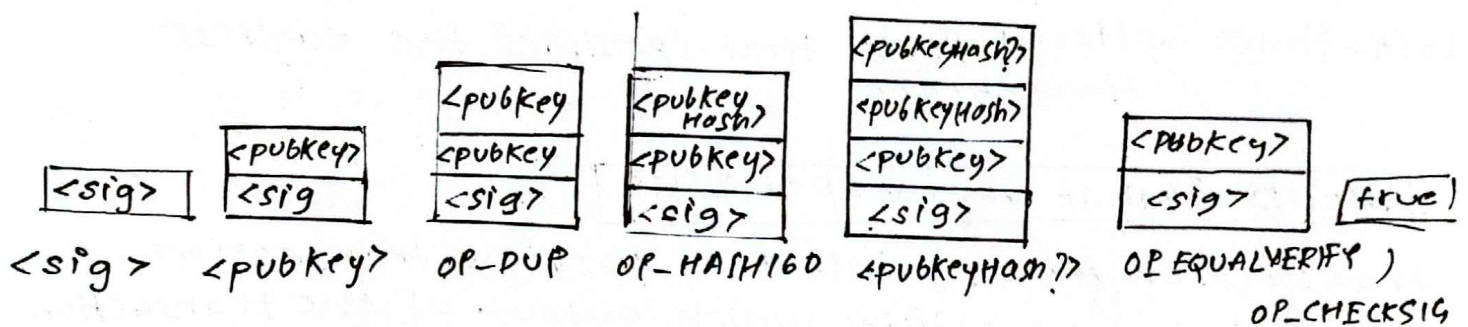
 OP-DUP
 OP-HASH160
 <pubkeyhash?>
 OP-EQUALVERIFY
 OP-CHECKSIG

Bitcoin script is stack based: instructions are pushed and pop values from a stack.

Stack based ensures that

1. Scripts are easy to verify
2. Miners won't get stuck running malicious infinite scripts
3. Rules are clear and deterministic.

EXECUTION OF A BITCOIN SCRIPT



Standard Script vs Theoretical Flexibility

In practice, 99.9% of transactions use just one script type: Pay-to-PublicKeyHash (P2PKH), which is the "classic" Bitcoin transaction (address + signature).

Other Scripts in Use

- MULTISIG
- Allow require m-of-n signature (eg. 2 of 3 keys must sig).
- Used in wallets, escrow, exchanges and Lightning Network funding

Pay-to-Script-Hash (P2SH)

- A powerful extension that lets you pay to the hash of a script
- This offloads complexity from the sender to the recipient

- Sender just pays to a short hash
- Recipients later reveals the full script + required data when redeeming coin.
- commonly used for MULTISIG, Lightning, segwit addresses etc

Proof-of-Burn (OP_RETURN)

- A script that can never be redeemed.
- Implemented by including OP_RETURN
- Effectively "destroy" the coins
- used for experiments, bootstrapping altcoins, or embedding arbitrary data in block-chain
- OP_RETURN <arbitrary data>
- Any transaction output with this script is provably unspendable

Pay to script Hash (P2SH)

Sender just pays to a short hash of the redeem script.

Later Receiver reveals the actual script + required unlocking data

Benefits

- simpler for senders

Efficiency: unspent transaction outputs (UTXOs) are smaller since only a hash is stored in output

complexity is moved to input script

Application of Bitcoin Scripts

Escrow Transactions (using MULTISIG)

Alice and Bob do not trust each other in a trade solution:- use of 2-OF-3 MULTISIG with Alice, Bob, and an arbitrator (Judy)

Alice Funds 2-OF-3 address

if all goes well: Alice + Bob sign $\rightarrow$ Bob gets paid

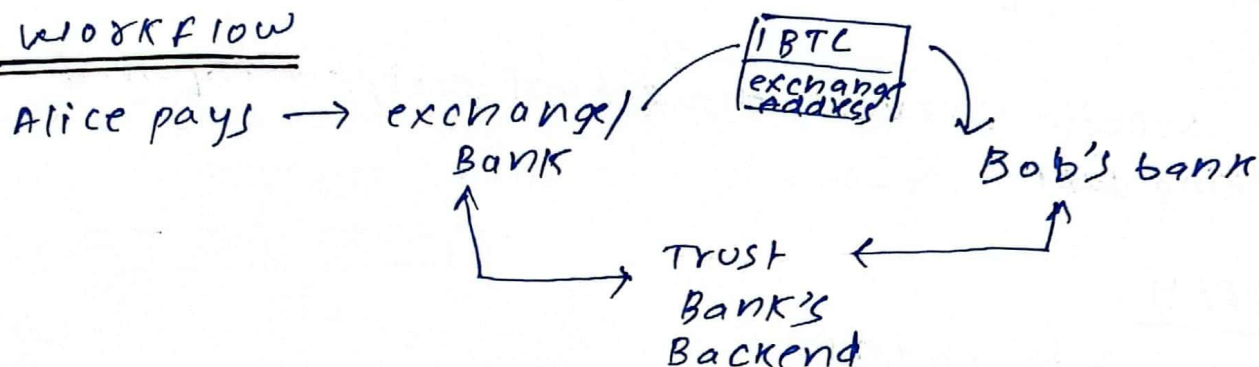
if dispute Arbitrator + one party sign $\rightarrow$ Funds returned to Alice.

GREEN Addresses

Bob wants fast payment but cannot (or won't) wait for blockchain confirmation

solution: Alice pays through her bank/exchange, which uses a green address

Workflow



Now no-one uses as services like Instawallet and Mt Gox (Bank) collapsed

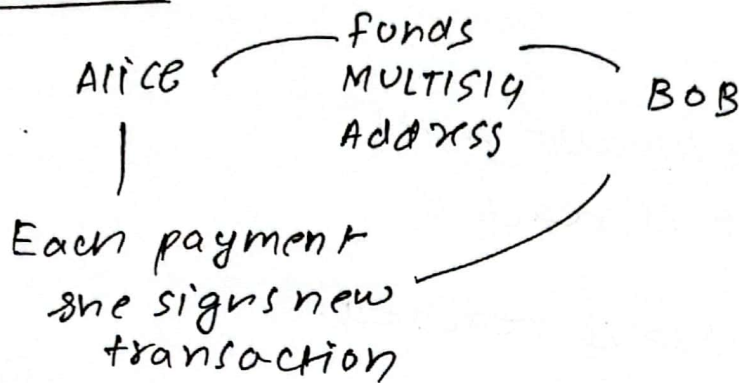
Not widely used anymore.

3. Efficient Micro-Payments

paying small amounts frequently (Alice pays Bob per minute @ some amount) creates too many transactions and high fees.

solution: use an off-chain micropayment channel with MULTISIG + incremental signed transaction.

Workflow

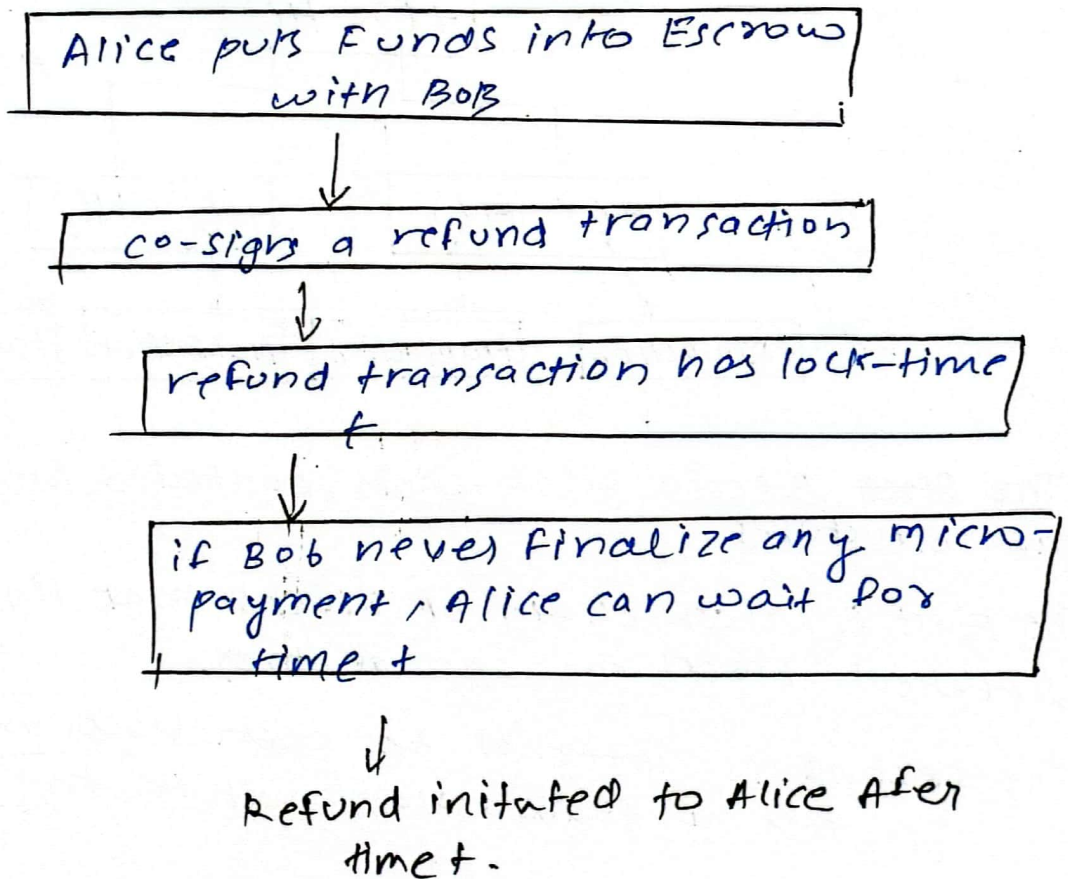


only final signed transaction is added to blockchain.

use Lock-time refund transaction, so Alice can reclaim her funds if Bob refuses to close the channel

Lock-time in Bitcoin

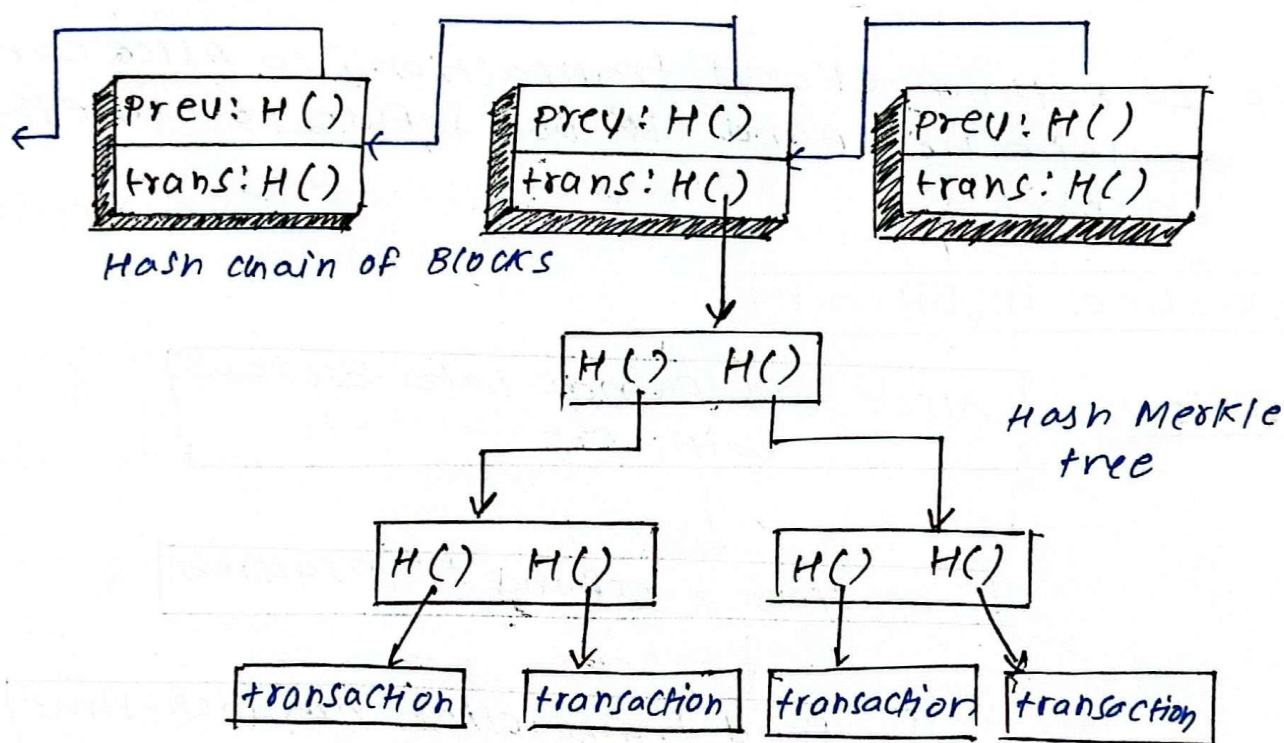
Workflow



What is lock-time in Bitcoin?

- metafield in a Bitcoin transaction
- $\text{lock_time} = 0$, transaction is included in the block immediately
- $\text{lock_time} > 0$
 - transaction becomes invalid until
 - a specific block nt is reached
 - specific UNIX timestamp is reached.

BITCOIN BLOCKS



The ~~Block~~ Bitcoin block-chain contains two different hash structures

The First is a hash chain of blocks that links the different blocks to one another

The second is internal to each block and is a merkle tree of transaction within the blocks.

Blocks are fundamental unit of consensus in Bitcoin

They combine

- [Hash chain of headers
- [Merkle tree of transaction

coinbase transaction is where new bitcoins are created + fees are distributed

Why ~~4~~ Transactions are grouped into Blocks?

- [Optimization
- [Efficiency
- [Verification